

第5章

Ajax及远程服务器数据处理技术

利用jQuery封装的Ajax技术，可以轻松实现客户端与服务端异步通信问题，从而帮助开发人员从烦琐的技术细节中解放出来，专心于业务层开发工作。

本章将详细介绍Ajax技术、JSON格式数据及远程服务器数据处理技术等知识。

5.1 Ajax技术概述

5.1.1 Ajax技术简介

- Ajax（Asynchronous JavaScript And XML，异步JavaScript 及XML），是一种应用JavaScript创建交互式[网页](#)应用的技术。通过在后台与服务器进行少量数据交换，Ajax可以使网页实现异步更新。这意味着可以在不重新加载整个网页的情况下，对网页的某部分进行更新。而传统的网页（不使用Ajax）要更新内容必须重新加载整个网页页面。

Ajax不是一种新的编程语言，而使用应用JavaScript在web浏览器与web服务器之间进行发送和接收数据的网页开发技术，是HTML、CSS及JavaScript等技术的组合。Ajax的核心是JavaScript的XmlHttpRequest对象。该对象是一种支持异步请求的技术。简而言之，XmlHttpRequest可以使用JavaScript向服务器提出请求并处理响应，而不阻塞用户。

在应用Ajax进行开发时，由于JavaScript的代码编写很大，通常使用jQuery编写。

5.1.2 Ajax技术的应用

1. 创建 XmlHttpRequest 对象

Ajax 技术的核心是使用 XmlHttpRequest 对象，使用 XmlHttpRequest 对象之前，需要实例化 XmlHttpRequest 对象。

```
xmlHttp = new XMLHttpRequest();
```

2. 配置 XMLHttpRequest 对象

XmlHttpRequest 对象创建以后，需要调用 open() 方法对 XmlHttpRequest 对象进行配置。

```
xmlHttp.open(method, url, true);
```

其中参数：

method 为提交方式：POST 或 GET；

url 为请求的数据源地址。

3. 发送请求并接收服务器的回应

xmlHttp.send(content);
其中参数：
contento 为指定要发送的数据。

4. XMLHttpRequest 对象的 readyState 属性、status 属性和 responseText 属性

(1) readyState 属性

XMLHttpRequest 对象的 readyState 属性用于返回 Ajax 的当前状态。其状态值有 5 种形式，其取值如表 5-1 所示。

readyState 属性值	说 明
0 表示“未初始化”状态；	已经创建一个 xmlhttprequest 对象，但是还没有初始化。
1 表示“已初始化”状态；	代 码 已 经 调 用 了 xmlhttprequest open() 方 法 并 且 xmlhttprequest 已经准备把一个请求发送到服务器。
2 表示“发送”状态；	已经通过 send() 方法把请求发送到服务器端，但是还没有收到响应。
3 表示“正在接收”状态；	已经接收到 http 响应头部信息，但是消息体部分还没有完全接收结束。
4 表示“已加载完成”状态；	响应已经被完全接收。

(2) status 属性

status 属性用于返回当前请求的 http 状态代码，常见的状态码如下所示：

- 200 - 服务器成功返回。
- 403 - 服务器禁止访问。
- 404 - 请求的网页不存在。

(3).responseText 属性

XMLHttpRequest 对象的.responseText 属性用于将响应信息作为字符串返回。

【例5-1】 Ajax对象发送和接收数据示例。

应用项目文件index.html的表单通过Ajax文件t5_1.js把表单中数据发送到IP地址为58.199.89.161的远程服务器www目录下的PHP文件t5_1.php中。t5_1.php再通过Ajax文件t5_1.js把数据回显到index.html的页面上。

(1) index.html代码:

```
<!doctype html>
<html>
<head>
  <meta charset="UTF-8">
  <script type="text/javascript" src="js/jquery-3.1.1.min.js" ></script>
  <script type="text/javascript" src="js/t5_1.js"></script>
</head>
<body>
  <p>用户名: <input type="text" id="txt-username"/>
  <p>密 码: <input type="text" id="txt-passwd"/>
  <p><input type="button" value="提交" onclick="btn_click();" />
  <div id="result"></div>
</body>
</html>
```

(2) Ajax文件 t5_1.js

```
function btn_click(){
    // (1) 创建XMLHttpRequest对象
    var xmlHttp = new XMLHttpRequest();
    // (2) 获取页面中输入的数据值
    var userName = document.getElementById("txt-username").value;
    var passWd = document.getElementById("txt-passwd").value;
    // (3) 配置XMLHttpRequest对象
    var url = "http://58.199.89.161/t10_1.php?username=" + userName
        + "&passwd=" + passWd; //58.199.89.161为远程服务器的IP地址
    xmlHttp.open("get", url, true);
    // (4) 设置回调函数
    xmlHttp.onreadystatechange = function(){
        if(xmlHttp.readyState == 4 && xmlHttp.status == 200){
            document.getElementById("result").innerHTML = xmlHttp.responseText;
            window.alert("数据返回: " + xmlHttp.responseText);
        }
    };
    // (5) 发送请求
    xmlHttp.send(null);
}
```


(3) PHP文件 t5_1.php

```
<?php
```

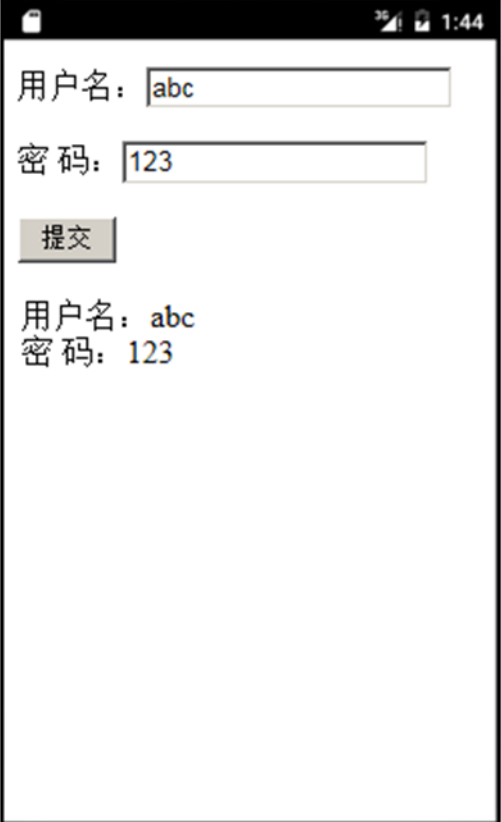
```
$name = $_GET["username"];
```

```
$pass = $_GET["passwd"];
```

```
echo "用户名: ".$name."<br>"; //用点.连接变量与字符串
```

```
echo "密 码: ".$pass."<br>";
```

```
?>
```



The screenshot shows a mobile browser interface. At the top, there's a status bar with signal strength, battery, and time (1:44). Below it, the page content includes a form with two input fields: "用户名:" (Username) containing "abc" and "密 码:" (Password) containing "123". A "提交" (Submit) button is positioned below the password field. After the button, the output of the PHP script is displayed: "用户名: abc" and "密 码: 123" on separate lines.

5.2 JSON数据

JSON（JavaScript Object Notation，JavaScript对象表示法）是一种轻量级的数据交换格式。

JSON采用完全独立于语言的纯文本格式，易于人们阅读和编写，同时也易于机器解析和生成（一般用于提升网络传输速率），因此，JSON成为网络传输中理想的数据交换语言。

5.2.1 JSON数据格式

JSON是基于JavaScript的一个子集，其数据格式非常简单。

1. JSON数据

JSON数据值可以是一个简单的字符串（String）、数值（Number）、布尔值（Boolean），也可以是一个数组或一个复杂的Object对象。

2. 用键-值对表示数据

JSON 数据的书写格式是：

键名(key) : 值(value) 。

键-值对的键名key必须是字符串，后面写一个冒号' : '，然后是值value，值value可以是字符串、数值、布尔值。

例如：

```
"firstName" : "John"
```

3. JSON对象

JSON对象可以包含多个键-值对，在花括号 ‘{ }’ 中书写，键-值对之间用逗号 ‘,’ 分隔。

例如：

```
{ "firstName":"John" , "lastName":"Doe" , "age":20 }
```

JSON对象的值也可以是另一个对象，例如：

```
{  
  "Name":"John" ,  
  "age": 20 ,  
  "hobby":"打篮球",  
  "friend":{"Name":"Suny" , "age":19 , "hobby":"看书"}  
}
```

4. JSON数组

JSON数组可以包含多个JSON数据作数组元素，每个元素之间用逗号','分隔，要求在方括号'[]'中书写。

例如：

```
var meber = [ "John" , 20 , "打篮球" ];
```

JSON数组的元素可以包含多个对象，例如：

```
var employees = [  
  { "firstName":"John" , "lastName":"Doe" },  
  { "firstName":"Anna" , "lastName":"Smith" },  
  { "firstName":"Peter" , "lastName":"Jones" }  
];
```

访问 JavaScript 对象数组中的第一项元素:

```
employees[0].lastName;
```

返回的值为: Doe

也可以修改其数据:

```
employees[0].lastName = "Jobs";
```

5. JSON文件

JSON 文件的文件类型是 ".json", JSON 文本的 MIME 类型是 "application/json"。

5.2.2 应用Ajax解析JSON数据

1. Ajax解析JSON对象数据

【例5-2】设有JSON对象{"ID":"1001","Name":"张大山","scor":92}，编写一个程序，将数据显示出来。

```
<script>
```

```
var json_data = {"ID":"1001","Name":"张大山","scor":92};
```

```
function myAjax()
```

```
{
```

```
    alert("学号: " + json_data.ID + "\n"
```

```
        + "姓名: " + json_data.Name + "\n"
```

```
        + "成绩: " + json_data.scor + "\n");
```

```
}
```

```
</script>
```

Ajax 解析 JSON 数据

```
<body>
```

```
    <div>
```

```
        <p><h1>JavaScript 解析 JSON 数据</h1></p>
```

```
        <p><button  onclick="myAjax();"><h2>解析 JSON 对象</h2></button></p>
```

```
    </div>
```

```
</body>
```



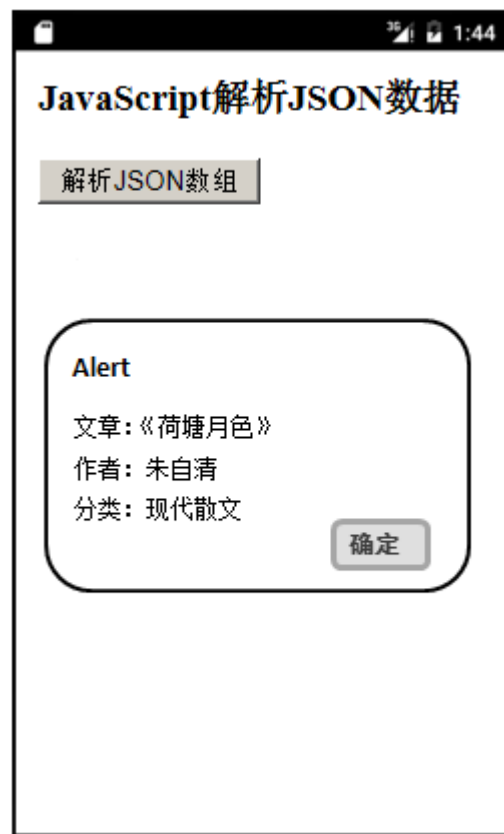
2. Ajax解析JSON数组数据

【例5-3】设有JSON数组["《荷塘月色》","朱自清","现代散文"],编写一个程序，将数据显示出来。

```
<script>
    var json_data = ["《荷塘月色》","朱自清","现代散文"];
    function myAjax()
    {
        alert("文章: " + json_data[0] + "\n"
            + "作者: " + json_data[1] + "\n"
            + "分类: " + json_data[2] + "\n");
    }
</script>
```

} Ajax 解析 JSON 数组

```
<div>
    <p><h1>JavaScript 解析 JSON 数据</h1></p>
    <p><button onclick="myAjax();"><h2>解析 JSON 数组</h2></button></p>
</div>
```



3. Ajax解析复杂的JSON数据

【例5-4】 设有数组与Object对象嵌套的复杂JSON数据

```
{  
    "sid":"1001",  
    "Name":"张大山",  
    "cla":[  
        {"km1":"大学英语 ","score1":90},  
        {"km2":"高等数学 ","score2":75},  
        {"km3":"程序设计 ","score3":95},  
    ]  
}
```

编写一个程序，将数据显示出来。

```

<script>
    var json_data = {
        "sid": "1001",
        "Name": "张大山",
        "cla": [
            {"km1": "大学英语 ", "score1": 90},
            {"km2": "高等数学 ", "score2": 75},
            {"km3": "程序设计 ", "score3": 95},
        ]
    };

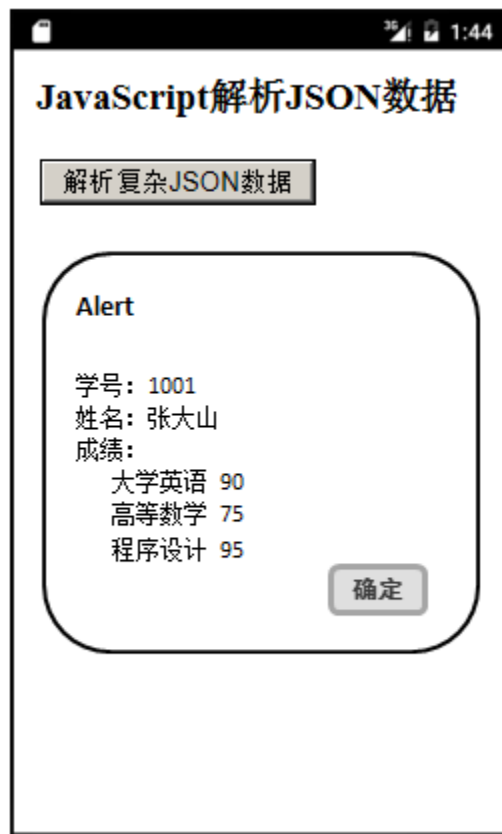
    function myAjax()
    {
        alert("学号: " + json_data.sid + "\n"
            + "姓名: " + json_data.Name + "\n"
            + "成绩: " + "\n"
            + "    " + json_data.cla[0].km1 + json_data.cla[0].score1 + "\n"
            + "    " + json_data.cla[1].km2 + json_data.cla[1].score2 + "\n"
            + "    " + json_data.cla[2].km3 + json_data.cla[2].score3 + "\n"
            );
    }
</script>

```

```

<div>
    <p><h1>JavaScript 解析 JSON 数据</h1></p>
    <p><button  onclick="myAjax();" ><h2>解析复杂 JSON 数据 </h2></button></p>
</div>

```



【例5-5】设有数组嵌套的复杂JSON数据

[

["《荷塘月色》","朱自清","现代散文"],

["《三国演义》","罗贯中","古典小说"],

["《神雕侠侣》","金庸","武侠小说"]

];

编写一个程序，将数据显示出来。

```

<script>
    var json_data = [
        ["《荷塘月色》", "朱自清", "现代散文"],
        ["《三国演义》", "罗贯中", "古典小说"],
        ["《神雕侠侣》", "金庸", "武侠小说"] ];

    function myAjax()
    {
        //应用 jQuery 元素选择器$("#app"), 将<p id="app">元素用 append() 追加显示。
        $("#app").append("文章: " + json_data[0][0] + "<br>");
        $("#app").append("作者: " + json_data[0][1] + "<br>");
        $("#app").append("分类: " + json_data[0][2] + "<br><br>");
        $("#app").append("文章: " + json_data[1][0] + "<br>");
        $("#app").append("作者: " + json_data[1][1] + "<br>");
        $("#app").append("分类: " + json_data[1][2] + "<br><br>");
        $("#app").append("文章: " + json_data[2][0] + "<br>");
        $("#app").append("作者: " + json_data[2][1] + "<br>");
        $("#app").append("分类: " + json_data[2][2] + "<br><br>");
        for(i=0; i<json_data.length; i++)
            $("#app").append(json_data[i]+ "<br>");
    }
</script>

```

```

<div>
    <p><h1>JavaScript 解析 JSON 数据</h1></p>
    <p><button onclick="myAjax();"><h2>解析 JSON 复杂数组</h2></button></p>
    <p id="app"></p>
</div>

```

JavaScript解析JSON数据

解析JSON复杂数组

文章：《荷塘月色》

作者：朱自清

分类：现代散文

文章：《三国演义》

作者：罗贯中

分类：古典小说

文章：《神雕侠侣》

作者：金庸

分类：武侠小说

《荷塘月色》，朱自清，现代散文

《三国演义》，罗贯中，古典小说

《神雕侠侣》，金庸，武侠小说

5.3 Ajax与PHP基础

5.3.1 PHP基础

1. PHP是什么

PHP即 Hypertext Preprocessor的缩写，意思是超文本预处理器，是一种通用开源的脚本语言。

2. 创建PHP网站

通常可以使用WampServer创建PHP网站。WampServer是一款集成了Apache Web服务器、PHP程序解释器以及MySQL数据库的软件包，在Windows操作系统下安装非常简便。

3. 第一个PHP程序

PHP程序以“<?php”标记开头，以“?>”标记结束，其基本格式如下面示例：

```
<?php
```

```
    echo ‘微笑着吃饭好帅啊’
```

```
    echo ‘真的好帅啊’
```

```
?>
```

4. PHP语法简介

PHP的变量声明格式为：

\$变量名；

例如：

```
<?php
```

```
    $x = 5;
```

```
    $a = "字符串";
```

```
    echo $x;    //显示变量的值
```

```
    echo "<br/>"; //换行
```

```
    echo $a;    //显示变量的值
```

```
?>
```

【例5-6】编写一个显示学生资料的PHP程序。

```
<?php
$name = '张大山';
$xh = 'swj14005';
$kc = 'Ajax';
$cj = 85;
?>
<html>
<meta charset="utf-8">
<body>
<h1>学生资料</h1> 
<table>
<tr>
<td>姓名: </td>
<td><?php echo $name ?></td>
</tr>
<tr>
<td>学号: </td>
<td><?php echo $xh ?></td>
</tr>
<tr>
<td>课程: </td>
<td><?php echo $kc ?></td>
</tr>
<tr>
<td>成绩</td>
<td><?php echo $cj ?></td>
</tr>
</table>
</body>
</html>
```



在页面中显示 PHP 变量值

在页面中显示 PHP 变量值

在页面中显示 PHP 变量值

在页面中显示 PHP 变量值

5.用表单提交数据

HTML表单用POST方式提交数据，需要使用\$_POST['数据名称']接收数据。

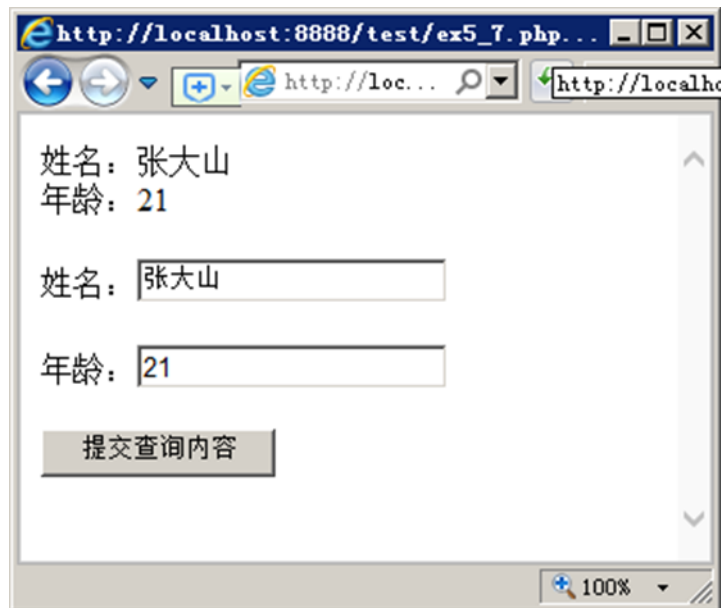
例如，从HTML表单向服务器以POST方式提交了2个数据，其数据名分别为name和age，则PHP的接收数据语句为：

```
<?php
    if($_SERVER['REQUEST_METHOD'] == 'POST')
    {
        $name = $_POST['name'];
        $age = $_POST['age'];
    }
?>
```

【例5-7】用表单提交数据并在同一文件中显示提交的内容。

```
<?php
    if($_SERVER['REQUEST_METHOD'] == 'POST')
    {
        $name = $_POST['name'];
        $age = $_POST['age'];
        echo "姓名: ".$name."<br>"; //用点. 连接字符串与变量
        echo "年龄: ".$age."<br>";
    }
?>
```

接收表单提交的数据



```
<form method="post" action="ex5_7.php">
    <p>姓名: <input type="text" name="name"/></p>
    <p>年龄: <input type="text" name="age"/></p>
    <p> <input type="submit"> </p>
</form>
```

5.3.2 jQuery 的Ajax 方法

通过jQuery的Ajax方法，能够使用HTTP Get和HTTP Post从远程服务器上请求文本、HTML、XML或JSON，同时能够把这些外部数据直接载入网页的被选元素中。在jQuery中，常用的Ajax操作方法如表所示。

表 5-2 常用的 Ajax 方法

方 法	说 明
<code>\$(selector).load(url, data, callback)</code>	把远程数据加载到被选的元素中
<code>\$.ajax(options)</code>	把远程数据加载到 XMLHttpRequest 对象中
<code>\$.get(url, data, callback, type)</code>	使用 HTTP GET 来加载远程数据
<code>\$.post(url, data, callback, type)</code>	使用 HTTP POST 来加载远程数据
<code>\$.getJSON(url, data, callback)</code>	使用 HTTP GET 来加载远程 JSON 数据文件
<code>\$.getScript(url, callback)</code>	加载并执行远程的 JavaScript 文件

1.用jQuery的\$.post()方法发送数据

在jQuery中使用POST方式异步提交数据，需要使用jQuery的\$.post()方法发送HTTP POST请求。

\$.post()方法的语法格式为：

\$.post(url [, data] [, callback] [, type])

参数意义说明：

url: 请求的HTML页面的URL地址。

data: 可选参数，发送到服务器端的数据。

callback: 可选参数，当请求成功时运行的回调函数。

type: 可选参数，服务器响应的数据类型。默认为服务器智能判断。

其中，回调函数callback的一般格式如下：

```
function(data, status)
{
    ...; //(1) data: 服务器端返回的数据内容，可以是Json、XML、HTML文档等
    ...; //(2) status: 服务器端返回的请求状态
}
```


【例5-8】 编写一个用户留言的页面，使用jQuery的\$.post()方法向远程服务器发送数据，并显示返回结果。

(1) 新建 ex5_8.html 文件，构建 form 表单，其主要代码如下：

```
<form method="post" action="">
    用户名: <input type="text" id="username" /></br><br/>
    留 言: <textarea id="content"></textarea><br/><br/>
    <input type="button" id="button" value="提交" /><br/><br/>
    <div id="responseText"></div>
</form>
```

(2) 编写按钮的 click 事件，将 form 表单中的用户名、留言内容作为参数传递给服务器端的 ex5_8.php 程序，代码如下：

```
$.post(
    "http://localhost/test/ex5_8.php", ← 第一个参数 url
    {
        username:$("#username").val(),
        content:$("#content").val()
    }, ← 第二个参数 data
    function(data) {
        $("#responseText").html("用户名: " +
            data.username + "<br/>留言内容: " +
            data.content); ← 第三个参数回调函数
    },
    "json"); ← 第四个参数 type，数据返回的格式为 JSON 数据
});
```

(3) 编写服务端 ex5_8.php 程序,接收 ex5_8.html 页面发送过来的数据,保存到 \$dataArray 数组中,之后使用 json_encode() 方法将数据转换成 json 对象后,返回给请求页面。PHP 程序 ex5_8.php 的代码如下:

```
<?php
    if($_SERVER['REQUEST_METHOD'] == 'POST')
    {
        $username = $_POST['username'];
        $content = $_POST['content'];
        $dataArray = array(
            "username" => $username,
            "content" => $content
        );
        $jsonStr = json_encode($dataArray);
        echo $jsonStr;
    }
?>
```

接收表单提交的数据

构建 Json 数组

将 Json 数组转换成对象

用户名: 张大山

留言: 到郊外春游很开心。

提交

用户名: 张大山
留言内容: 到郊外春游很开心。

2. 用jQuery的\$.get()方法发送数据

在jQuery中使用GET方式提交请求数据，需要使用jQuery的\$.get()方法发送HTTP GET请求。

\$.get()方法的语法格式为：

\$.get(url [, data] [, callback] [, type])

【例5-9】 编写一个用户登录的页面，使用jQuery的\$.post()方法向远程服务器发送数据，并显示返回结果。

(1) 新建ex5_9.html文件，核代码如下：

```
<script>
$(document).ready(function() {
    $("#button").click(function() {
        $.get(
            "http://localhost/test/ex5_9.php",
            {
                sid:$("#sid").val(),
                username:$("#username").val(),
                email:$("#email").val()
            },
            function(data, status) {
                $("#responseText").html("序号: " +
                    data.sid + "<br/>用户名: " +
                    data.username + "<br/>邮箱: " +
                    data.email + "<br/>返回状态: " +
                    status);
            },
            "json");
    });
});
</script>
```

The diagram illustrates the four parameters of the `$.get()` method:

- 第一个参数 url**: Points to the URL `"http://localhost/test/ex5_9.php"`.
- 第二个参数 data**: Points to the data object `{ sid:$("#sid").val(), username:$("#username").val(), email:$("#email").val() }`.
- 第三个参数回调函数**: Points to the callback function `function(data, status) { ... }`.
- 第四个参数 type, 数据返回的格式为 JSON 数据**: Points to the `"json"` parameter.

A bracket on the right side groups all four parameters under the label **\$.get() 方法**.

(2) 编写服务端ex5_9.php程序，接收ex5_9.html页面发送过来的数据。

```
<?php
```

```
if($_SERVER['REQUEST_METHOD'] == 'GET')
```

```
{
```

```
    $sid = $_GET['sid'];
```

```
    $username = $_GET['username'];
```

```
    $email = $_GET['email'];
```

接收表单提交的数据

```
    $dataArray = array(
```

```
        "sid" => $sid,
```

```
        "username" => $username,
```

```
        "email" => $email );
```

构建 Json 数组

```
    $jsonStr = json_encode($dataArray);
```

将 Json 数组转换成对象

```
    echo $jsonStr;
```

```
}
```

```
?>
```

序号: a1001

用户名: 李晓丽

邮箱: lixiaoli@163.com

提交

序号: a1001
用户名: 李晓丽
邮箱: lixiaoli@163.com
返回状态: success

3. 用\$.getJSON()方法加载JSON文件

在jQuery中，\$.getJSON()方法用于加载JSON格式的数据文件。例如，要加载test.json文件的数据，其代码如下：

```
$("#button").click(function( ){  
    $.getJSON("test.json", 回调函数 );  
})
```

其中回调函数为：

```
function(data){  
    $.each(data, function(数组的索引, 返回的数组元素)  
    {  
        ...; //服务器端返回的数据中包含的数组元素内容  
    });  
}
```

在回调函数中，使用\$.each()方法遍历返回的数据data，取出数组各元素的值。

【例5-10】 编写程序，读取JSON格式的数据文件。

(1) 在服务器端新建JSON格式的数据文件ex5_10.json:

```
[
  {"title": "《荷塘月色》", "author": "朱自清", "type": "现代散文"},
  {"title": "《三国演义》", "author": "罗贯中", "type": "古典小说"},
  {"title": "《神雕侠侣》", "author": "金庸", "type": "武侠小说"}
]
```

(2) 新建ex5_10.Html文件，读取JSON格式的文件ex5_10.json中的数据，其核心代码：

```
<script>
```

```
$(document).ready(function() {
```

```
  $("#button").click(function() {
```

```
    $.getJSON(
```

```
      "http://localhost/test/ex5_10.json",
```

```
      function(data) {
```

```
        var htmlStr = "";
```

```
        $.each(data, function(ex5_10, info) {
```

```
          htmlStr += "文章: " + info['title'] + "<br/>";
```

```
          htmlStr += "作者: " + info['author'] + "<br/>";
```

```
          htmlStr += "分类: " + info['type'] + "<br/><br/>";
```

```
        });
```

```
        $("#json").html(htmlStr);
```

```
      }); //$.getJSON()_end
```

```
    }); //click()_end
```

```
  }); //ready()_end
```

```
</script>
```

遍历 Json 数组

在 id="json" 的<div>中显示数据

读取JSON文件

读取数据

文章：《荷塘月色》

作者：朱自清

分类：现代散文

文章：《三国演义》

作者：罗贯中

分类：古典小说

文章：《神雕侠侣》

作者：金庸

分类：武侠小说